UNIVERSITY OF TÜBINGEN · LLM INFERENCE OPTIMIZATION SEMINAR · SUMMER 2026

Long-Context Inference

Ring Attention | Striped Attention | Native Sparse Attention | SwiftKV | StreamingLLM

Kavyanjali Agnihotri

Why 1M Tokens?

> Context windows have grown from roughly **512** tokens in 2018 to **2K**, then **100K–128K**, and now **1M** tokens at the frontier. This talk explains why that jump matters and how the memory cost is managed.

Document Analysis

Legal contracts, research papers, full codebases in a single pass. Chunking creates seam artifacts; full context does not.

Agentic Loops

Multi-step agent traces accumulate tool outputs, memory scratchpads, and prior turns. Context grows with task complexity.

Multimodal

A single high-res video frame or interleaved image sequence easily reaches tens of thousands of tokens.

> **Scale:** 128K tokens ≈ a full-length novel (96K English words). 1M tokens ≈ 750 words.

KV Cache: What It Is and Why It Costs

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

- **Q** — what this token is looking for
- **K** — what every other token contains
- **V** — the actual content of those tokens

1. During autoregressive generation, each transformer layer computes key and value vectors from the input, which are cached in GPU memory to avoid recomputation at each step.

2. During decoding, the per-step complexity lowers from O(n²) to O(n), but at the cost of storing all those tensors.

KV Cache per token is a fixed constant per model architecture

$$\text{Total KV size} = \underbrace{2 \times L \times H_{kv} \times d \times \text{bytes}}_{\text{KV Cache per token}} \times n \times b$$

L=layers · H=KV heads · d=head dim · n=tokens (sequence length) · b=batch size

Llama-3.1-8B: 32 layers · 8 KV heads · head dim 128 · FP32 → **0.25 MB per token**



2.5

GB

10,000 tokens

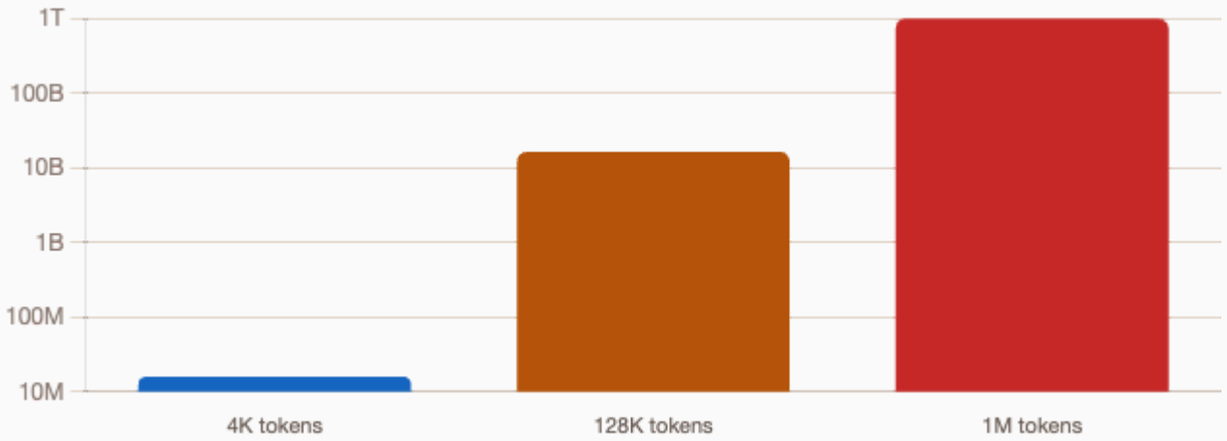
0.25 MB × 10,000 tokens =
2.5 GB

Sequence	KV Cache	Note
1,000	0.25 GB	Fits easily
10,000	2.5 GB	Comfortable
128,000	32 GB	Full H100
1,000,000	**250 GB**	4× H100s, KV only

Prefill: The Quadratic Wall

Prefill ingests the full prompt before generating any output.

Attention is $O(n^2)$: every token attends to every other token.



Attention score computations per head per layer (log scale)

Context	Ops (per head/layer)	Relative
4K tokens	16M	1×
128K tokens	16.4B	1,024×
1M tokens	1T	62,500×

Real Impact

A 1M-token prompt can take 30+ minutes to prefill on hardware that handles 4K tokens in under a second.
Prefill is compute-bound (unlike decoding, which is memory-bandwidth-bound).
Quadratic scaling makes it the dominant bottleneck.

Two Paths to Efficient Long-Context Inference

Lossless Exact

Mathematically identical output to full attention. Distribute the work, not reduce it.

- Ring Attention: sequence sharded across a GPU ring
- Striped Attention: causal-aware interleaved partition
- Context Parallelism in vLLM, SGLang

Memory savings: linear with device count N

Lossy Approximate

Compute a subset of attention. Trade small accuracy loss for large efficiency gain.

- Natively Trainable Sparse Attention (NSA): hierarchical sparse attention (DeepSeek)
- SwiftKV: skip later prefill layers
- StreamingLLM: attention sinks + sliding window

Speedup: 2×–12× depending on context and method

In practice: lossless methods keep the full sequence in memory; lossy methods reduce per-token computation. Production systems combine both.

PART I

Lossless Approaches

Context Parallelism: exact attention distributed across devices

Context Parallelism: Split Sequence, Share KV

By splitting the sequence across GPUs, each device holds only a fraction of the KV cache. Combined correctly, the result is mathematically exact to full attention.

Sequence split across N=4 GPUs:

$$Q = [Q_1, Q_2, Q_3, Q_4], \quad K = [K_1, K_2, K_3, K_4]$$

Memory per GPU:

$$\frac{\text{Total KV}}{N}$$

Concrete Savings (Llama-3.1-8B, 128K)
1 GPU: 32 GB — impossible once weights are loaded
4 GPUs: 8 GB each — fits comfortably
8 GPUs: 4 GB each — room for larger batches

The Global Softmax Problem

GPU 1 holds Q_1 and K_1 but true global attention needs all keys. Softmax requires the sum of exponentials across the entire sequence — GPU 1 can't compute it with only K_1.

$$\text{Out}_1 = \text{softmax}\big(Q_1[K_1^\top, K_2^\top, \ldots]\big)[V_1, V_2, \ldots]$$

The Solution: Attention between a query block and any set of KV blocks can be computed in any order — as long as softmax statistics (running max, sum of exp, weighted output) are correctly merged and rescaled after each block.

Ring Attention LOSSLESS

Idea: KV blocks around a GPU ring. Accumulate partial softmax statistics using an online algorithm that can merge blocks exactly and in any order.

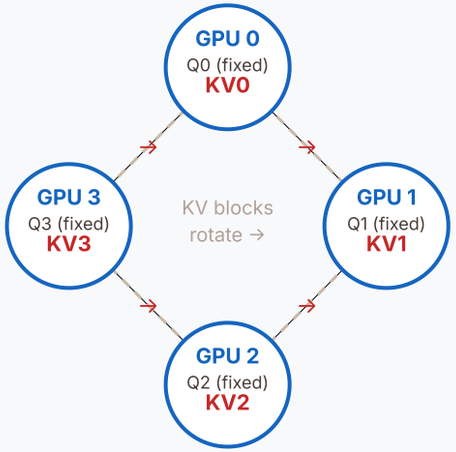
Algorithm

1. Split sequence across N GPUs. Each GPU holds a fixed query block Q_i.

2. Over N−1 rounds, KV blocks rotate clockwise: GPU i sends to GPU i+1, receives from GPU i−1 simultaneously.

3. Each GPU computes blockwise attention against the current KV block, accumulating partial softmax statistics.

4. After N−1 rounds, every GPU has seen all KV blocks. Output is bit-for-bit exact.



Step → Reset Round 0 of 3

Q blocks (blue) stay fixed · KV blocks (red) rotate clockwise

The Sequential Constraint: GPU cannot finalize its output until all N−1 KV blocks have rotated through. Round k feeds into round k+1. This is the fundamental latency cost of Ring Attention.

Ring Attention: Online Softmax

Why the Math is Exact

Softmax scores depend on the global maximum logit m^*. When a new KV block arrives with a higher max m', we rescale all prior partial sums by $\exp(m_{\text{old}} - m_{\text{new}})$. This corrects the shifted denominator exactly — no approximation.

Variables

- m: local maximum logit found so far in this block ($\max(x_i)$)
- x_i: raw attention score (logit) for a single token i.e. $Q \cdot K^T$
- s: local sum of exponentials
- v_i: value vector corresponding to the token
- a_i: local softmax probability for token i
- o: local attention output
- s_{new}: updated global denominator
- o_{new}: updated global attention output

Standard softmax cannot be split. Online softmax maintains running statistics that can be merged in any order with zero error.

Each device tracks:

$$m \leftarrow \max(\text{logits}), \quad s \leftarrow \sum e^{x_i - m}, \quad o \leftarrow \sum a_i v_i$$

When new KV block arrives with its own statistics (m', s', o') and if m' is the new global max m^*, then:

$$s_{\text{new}} = e^{m - m'} \cdot s + s'$$

$$o_{\text{new}} = \frac{e^{m - m'} \cdot s \cdot o \; + \; s' \cdot o'}{s_{\text{new}}}$$

Result: Output is bit-for-bit identical to full single-device attention. The only cost is KV block communication overhead between GPUs.

Ring Attention: Communication Overlap and Memory

Communication Overlap: While GPU i computes on the current KV block, it simultaneously sends that block to GPU i+1 and receives the next from GPU i−1. If compute time ≥ transfer time, there is zero communication overhead.

Block size c, hidden dim d, FLOP/s F, bandwidth B:

$$\frac{\text{Compute time}}{\text{Transfer time}} = \frac{\frac{4dc^2}{F}}{\frac{4cd}{B}}$$

Overlap when: $c \geq \frac{F}{B}$

Compute scales as c², communication as c — easy to satisfy in practice.

Memory Requirements: Each host (GPU) needs to hold 6 blocks: 1 query block (stationary) + 2 current K,V blocks + 2 incoming K,V blocks (buffered) + 1 output accumulator

Total = 6bch bytes

Independent of sequence size; b: batch size, c: block size, h: hidden dimension

Scaling Law

If a model supports context s on 1 GPU, Ring Attention supports N·s on N GPUs with identical flop utilization. Demonstrated beyond 100M tokens without approximation.

Striped Attention: Fixing Causal Workload Imbalance LOSSLESS

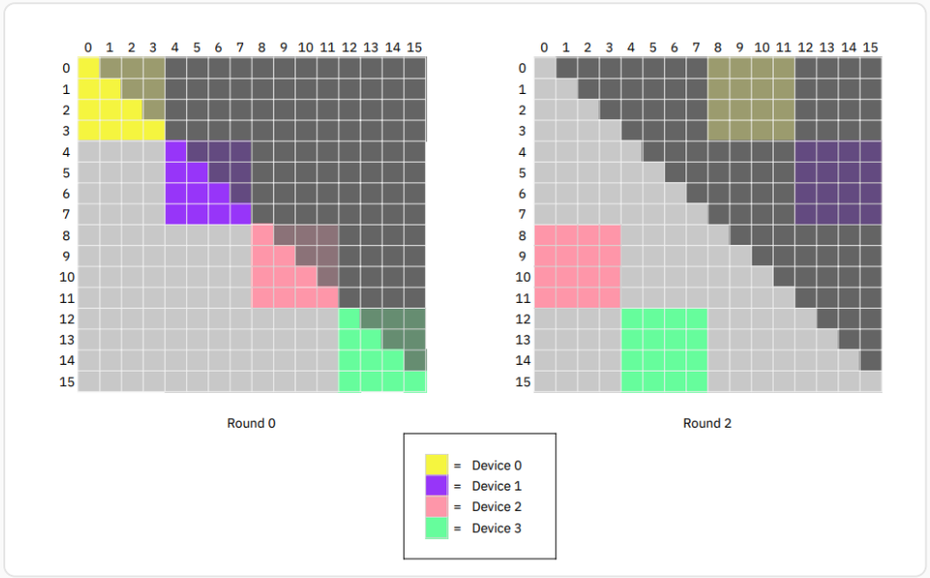
Causal attention masks all future tokens — theoretically halving total ops. But Ring Attention with contiguous blocks cannot exploit this due to workload imbalance.

$$\text{CausalAttn}(Q, K, V) = \text{softmax}(QK^\top + C)\,V$$

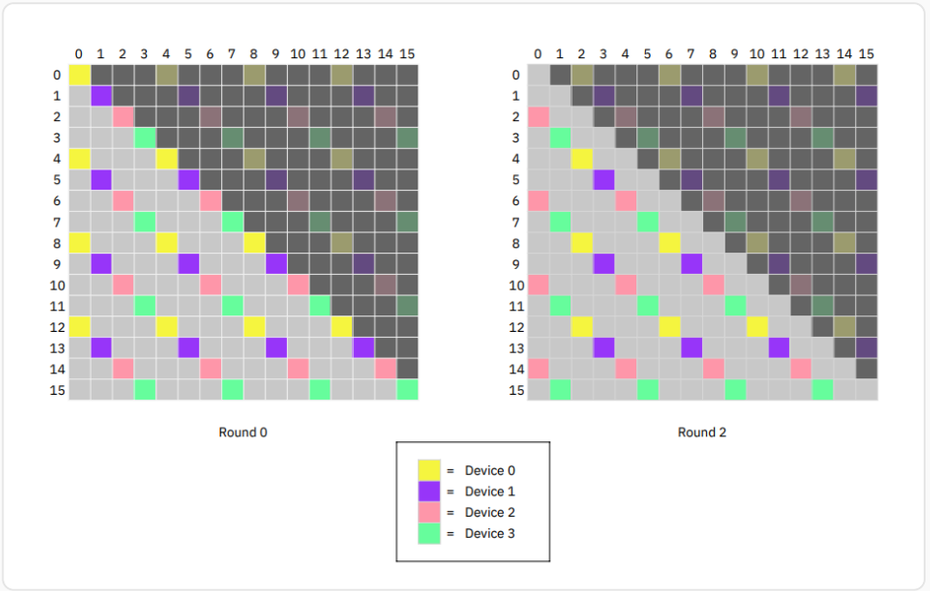
C: upper triangle = −∞ (softmax → 0, skipped)

The Imbalance

With contiguous blocks, some rounds give GPU A entirely masked (zero work) while GPU B gets entirely unmasked (full work). GPUs sync each round, so the slowest sets the pace — the 2× savings disappears.



(a) Ring Attention — Round 2 shows some GPUs with entirely masked (grey) blocks



(b) Striped Attention — Every GPU maintains balanced half-filled patterns every round

Figure 4, Brandon et al. 2023. The rows are the "Queries" (the current tokens being processed) and the columns are the "Keys" (the past tokens they are looking at). Colors = which GPU computes each query/key pair. Grey = masked by causal mask. Striped eliminates idle rounds entirely.

Striped Attention: Round-Robin Token Assignment

The Fix: Assign Tokens by Position mod N

Tokens are interleaved across devices instead of assigned in contiguous blocks. For N=4:

GPU 0
tokens 0, 4, 8, 12, ...

GPU 1
tokens 1, 5, 9, 13, ...

GPU 2
tokens 2, 6, 10, 14, ...

GPU 3
tokens 3, 7, 11, 15, ...

Every GPU holds tokens distributed uniformly across the full sequence — causal mask is roughly half-filled every round.

Workload per device: $\frac{c(c \pm 1)}{2}$ instead of worst-case c^2

Why Exact

Attention is permutation-equivariant in the key-value dimension. Permute tokens before splitting, unshuffle the output after — identical to full causal attention.

Hardware	Sequence	Speedup over Ring
8× A100	256K	1.45×
16× TPUv4	786K	1.65×

Trend

Speedup grows with sequence length and degree of sequence parallelism. At 786K tokens on 16 TPUs, 1.65× translates directly to reduced training cost.

PART II

Lossy Approaches

Reduce computation itself rather than distributing it

Native Sparse Attention (NSA) DEEPSEEK 2025

First sparse attention deployed in production at scale (DeepSeek-V3.2).
Unlike prior methods that bolt on sparsity after training, NSA is trained from scratch — the model learns *which* tokens matter.

Why Prior Sparse Methods Fail

- **Train/Inference Mismatch:** Bolting sparsity onto a dense model at inference time carries architectural bias and limits accuracy.
- **Hardware Inefficiency:** Random memory access destroys *arithmetic intensity*. GPUs stall while fetching fragmented data instead of doing math.

Arithmetic Intensity: Ratio of math operations to memory fetches, dictates what bottlenecks your GPU:

- **Compute-bound (High Intensity / Prefill):** GPU has enough data and is maxing out its processors. Happens during LLM training and prefill, where large batches are processed at once.
- **Memory-bound (Low Intensity / Decoding):** GPU finishes math instantly and sits idle waiting for data to load. Happens during LLM decoding, because generating just one single token requires fetching the entire KV-cache from memory.

NSA splits attention into three parallel branches:

1. Compression (Coarse): Token blocks compressed into single representations by a learnable MLP with overlapping strides. Panoramic view of the full context at low compute cost.

2. Block Selection (Fine): Compression scores reused to rank uncompressed blocks. Top-k selected at full resolution. Blockwise contiguous reads, FlashAttention friendly.

3. Sliding Window (Local): Fixed window of recent tokens in an isolated branch: prevents the model taking a "local shortcut" and skipping the global branches during training.

NSA: Architecture at a Glance

Output Merge via Learned Gating

Three branch outputs combined by a learned gate. All heads selecting identical blocks is not just algorithmic: it is a hardware prerequisite that keeps KV reads contiguous and eliminates redundant memory transfers in the custom kernel.

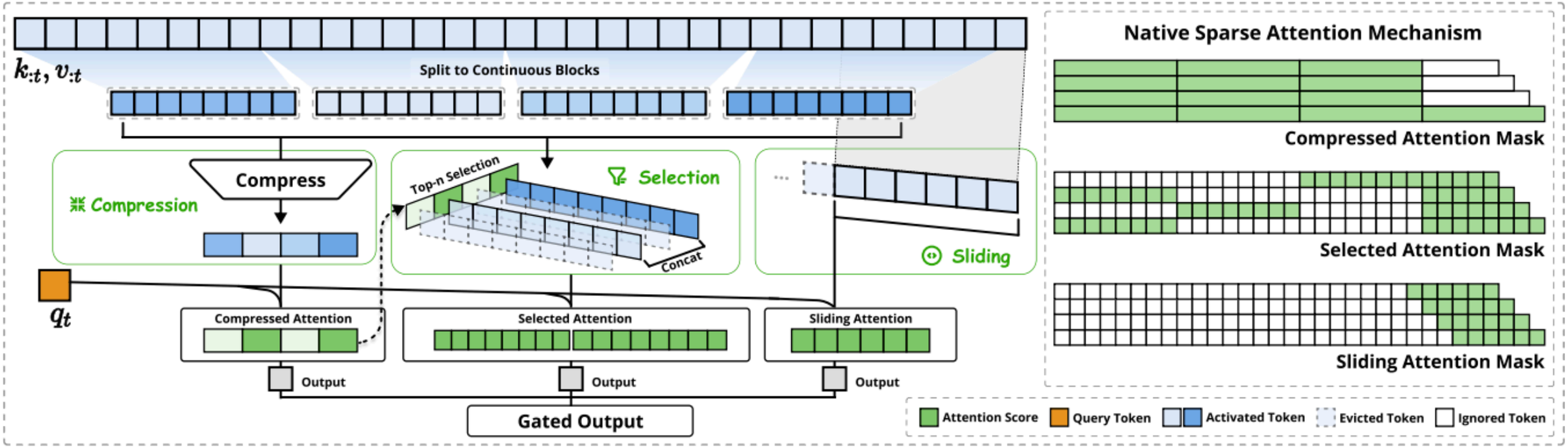


Figure 2, Yuan et al. 2025. Three parallel branches: compressed (coarse global), selected (important exact blocks), sliding window (local). Green tiles = computed; white tiles = skipped. All three outputs are merged by a learned gate.

NSA: Group-Centric Kernel and Results

Why Standard FlashAttention Fails? FlashAttention streams contiguous KV blocks. Sparse selection needs disjoint fragments for different query positions and scattered reads destroy arithmetic intensity.

NSA's Group-Centric Kernel (Triton)

Loading Strategy: Instead of grouping by timestamp (all queries at similar positions), NSA groups by GQA head group (all query heads that share a KV-cache) and loads all heads at once, then fetches the contiguous KV blocks they all share. Zero redundant transfers.

Algorithmic Constraints are strict hardware prerequisites designed to maximize memory efficiency.
Blockwise selection ensures the GPU fetches data in large, contiguous chunks, while **per-group aggregation** forces all heads in a group to request those exact same chunks, preventing scattered memory reads.
Together, these allow the **custom kernels** to process it at near-FlashAttention speeds alongside the naturally contiguous compression and sliding window branches.

Metric (64K context)	NSA vs Full Attn
Training throughput	2-4× faster
Decoding speed at 64K	11.6× faster
General benchmarks	matches or exceeds
Long-context retrieval	perfect accuracy
Chain-of-thought math	exceeds Full Attn

Results

Sparse patterns learned natively during pretraining: the model discovers which tokens genuinely matter, reducing attention noise.

SwiftKV: Later Layers Add Less SNOWFLAKE 2024

The Observation

As tokens travel through layers, their hidden states: each token's internal vector representation stop changing significantly. By the midpoint of the model, each token's representation is already close to its final-layer state.

Standard prefill (when the initial prompt is processed)
- all L layers run for every token:

$$h^{(1)} \to h^{(2)} \to \cdots \to h^{(L)}$$

SwiftKV — project KV for later layers from pivot layer output $h^{(l)}$:

$$h^{(l)} \xrightarrow{\text{project}} \text{KV}^{(l+1)}, \ldots, \text{KV}^{(L)}$$

What Gets Skipped During Prefill

For prompt tokens in layers l+1 to L: attention, MLP, Q projection, and output projection are all skipped. Only the K and V projections run, using $h^{(l)}$ as input.
* Decoding runs all layers unchanged to ensure high reasoning quality.

AcrossKV: Optional Memory Savings

While SwiftKV primarily saves compute time, its AcrossKV feature also significantly reduces memory usage. Normally, **skipped layers** still require their own stored KV cache. AcrossKV solves this by calculating **a single KV cache and sharing it across consecutive layers**. This shrinks the overall memory footprint, making it highly effective for long-context tasks where memory is the primary bottleneck.

SwiftKV: Recovery via Lightweight Distillation

Knowledge Distillation

The architectural change initially drops accuracy.
Recovery: freeze the original model as "teacher," fine-tune only the KV projection weights of skipped layers (less than 10% of total parameters) to match the teacher's output logits.
* Completes in hours, not weeks since trained on fewer than 1B tokens.

Zero Infrastructure Changes

SwiftKV only changes which data feeds into which projection matrix, not the operations themselves. Drops into vLLM and SGLang without custom kernels or modified serving stacks.

Metric	Result
Prefill compute reduction	25–50%
Aggregate throughput	up to 2×
Time per output token	60% lower
Normalized throughput	560 TFlops/GPU
Llama-3.1-70B tokens/sec	16K tokens/s
Benchmark quality loss	<2%
Model size range tested	3B to 405B

Best Fit

RAG and summarization: long prompts, short outputs. Prefill dominates total time.

Open source: Arctic Inference plugin for vLLM from Snowflake · Production-deployed in Arctic model serving

StreamingLLM: Why Window Attention Fails ICLR 2024

The Goal: Infinite Streaming

LLMs are needed for long-running conversations and streaming applications, but KV cache grows linearly with sequence length — unsustainable. The question: can we keep memory constant without losing coherence?

Two Failed Fixes

Window Attention: Keep only the most recent N KV states — constant memory. But once the sequence exceeds cache size, perplexity explodes. Position embeddings no longer match training; the model collapses.

Sliding Window + Recomputation: Rebuilds KV states for recent tokens at each step. Strong quality, but the quadratic recomputation cost makes it 22× slower than needed so impractical for real streaming.

The Root Cause: Attention Sinks

Both approaches miss a deeper problem. Softmax requires attention scores to sum to 1. When no token is a strong match, the model "dumps" spare attention onto the initial tokens as a "sink", even if those tokens carry no semantic relevance.
Evicting these sinks removes the dump site and destabilizes softmax.

StreamingLLM: Attention Sinks and the Fix

The Attention Sink Phenomenon

Transformers assign disproportionately high attention weight to the first few tokens in every sequence, regardless of their content. This is not a curiosity, it is load-bearing. Softmax must distribute probability mass somewhere; initial tokens become the default dump site.

Key experiment: Replace the first 4 tokens with "\n" (newline). The model still concentrates attention on them. It is the *absolute position* that matters, and not the content. This rules out semantic explanations and confirms the sink is positional.

Why This Matters

Any window attention scheme that evicts the initial tokens removes the sink. Without a place to dump spare attention, softmax becomes unstable and model quality collapses; explaining the window-attention failure exactly.

The Fix: Keep the Sinks

Permanently retain the first 4 tokens (attention sinks) in the KV cache alongside a rolling window of the most recent tokens. When new tokens arrive, evict only from the rolling window and never the sinks. Memory stays constant; softmax stays stable.

StreamingLLM: Solutions, Position Re-Indexing and Results

Two Solutions

- **Zero Sink (SoftMax-off-by-One):** Alters softmax so scores don't strictly sum to one ($e^{x_i} / (1 + \sum e^{x_j})$). This provides partial relief, the model still relies on other initial tokens as attention sinks. --*OR*--
- **Sink Token:** A dedicated, trainable token that absorbs excess attention. This is the *most effective* approach, stabilizing attention and improving overall model perplexity.

Position Re-Indexing

Assign position indices relative to *cache slots*, not original sequence positions. The model permanently believes it is reading a short document within its training window — it never hits the training-length limit, enabling truly infinite generation.

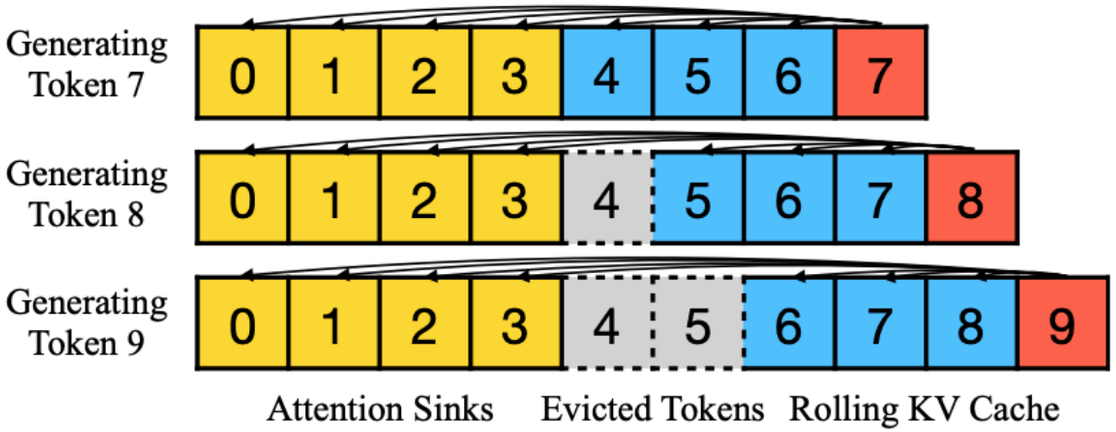


Figure 4, Xiao et al. 2023. KV cache = attention sinks (first 4 tokens, always retained) + rolling window. Position indices are relative to cache slots.

Results

Sinks serve as a mathematical dump for spare attention. The model ignores the artificial positional distance between sinks and recent tokens, as long as recent tokens maintain correct relative spacing. This enables contexts of **4M+ tokens** with a fixed KV cache, a **22.2× speedup** over sliding window recomputation, with no fine-tuning required across Llama-2, MPT, Falcon, and Pythia.

PART III

Industry Deployment

How production systems combine these ideas

SGLang: Chunked Pipeline Parallelism

Scenario: Serving a 100-layer model across 4 servers on a 100K-token prompt — how do you keep all servers busy?

1. Pipeline Parallelism (PP)

Layer groups split across servers over slow inter-server links. Server 1: layers 1–25, Server 2: 26–50, etc. Naive PP: Server 1 processes the full 100K prompt while Servers 2–4 sit completely idle. This is the pipeline bubble.

2. Context Parallelism (CP): Within a Server

Using the ultra-fast NVLink cables inside Server 1, CP splits the prompt itself: GPU A takes tokens 1–25,000, GPU B takes 25,001–50,000, and they compute Layer 1 at the exact same time by passing data in a continuous circle.

3. Chunked PP: Across Servers

Slice the prompt into 4K-token chunks. Server 1 processes chunk 1 through layers 1–25, passes it to Server 2, immediately picks up chunk 2. By the time chunk 1 reaches Server 4, all four servers work simultaneously on different chunks.

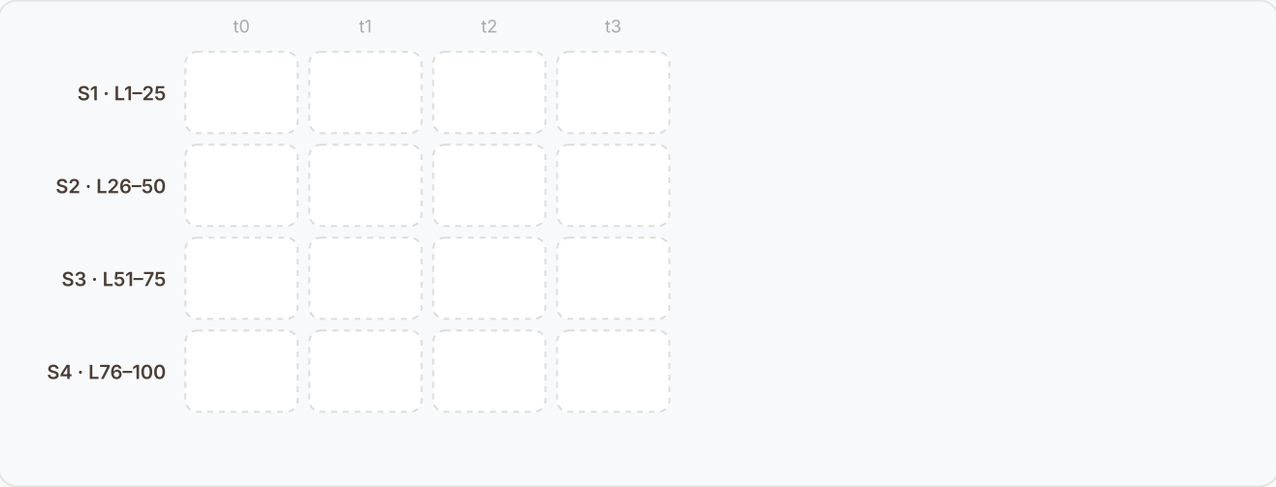
4. Hybrid CP × PP

n GPUs inside Server 1 use Context Parallelism to simultaneously compute the first 4,000-token chunk across Layers 1–25, and then use Chunked Pipeline Parallelism to pass that finished chunk to Server 2 over the slower network, instantly grabbing the next chunk. This hybrid strategy perfectly matches the software to the hardware, ensuring the GPUs never OOMs or idle.

Watch the Bubble Disappear: PP vs CP vs Both

Same 4-server, 100-layer setup as before. Toggle each strategy independently and step through time to see server utilization change.

Chunked PP: OFF Context Parallelism: OFF



Step → ▶ Play Reset Ready — 4 steps total

Naive PP: the whole prompt moves through one server at a time. Only 1 of 4 servers is ever busy, the other three sit idle. This is the pipeline bubble.

Click the toggles, then Step or Play through time. Colored = server busy on a chunk, grey = idle. Ring icon = GPUs inside that server splitting work via context parallelism.

vLLM vs SGLang: The Current Tension

vLLM: Betting on Context Parallelism

Ring and Striped Attention as the primary strategy. Helix (NVIDIA, 2025) proposes replacing the ring with All-to-All communication: every GPU sends its partial attention output and log-sum-exp factor to every other GPU simultaneously, reducing per-round latency.

- Ring/Striped enabled by default for long-context serving
- Day-0 vLLM support for DeepSeek NSA

SGLang: Betting on Pipeline Parallelism

Chunked PP favors model-layer sharding across servers. Maps better to cluster topologies where inter-server bandwidth is constrained relative to intra-server NVLink speed.

- CPP eliminates pipeline bubbles at long context
- CP within servers over NVLink remains fast

The Convergence

Both teams acknowledge the future is hybrid CP × PP. The debate is which dimension to optimize first given today's hardware. NVLink makes CP cheap within a server; InfiniBand makes PP cheaper across servers.

Summary

Method	Loss	Gain	Best Workload
Ring Attention	None	N× memory	Training, exact inference
Striped Attention	None	1.65× over Ring	Causal LM training
NSA	Minimal	9–12× at 64K	Pretraining, long decode
SwiftKV	<2%	2× throughput	RAG, summarization
StreamingLLM	Moderate	22× vs recompute	Infinite streaming

In practice: lossless methods fit the full sequence in memory; lossy methods reduce per-token computation. Production systems combine both.

Open Problems

- Optimal CP × PP hybrid scheduling
- NSA trained on one general mix, but retrieval, code, etc may want different sparsity emphasis
- SwiftKV: optimal layer cutoff per model
- Helix All-to-All vs ring topology tradeoffs
- Attention Sink only studied standard decoder-only causal LMs

The Goal: Achieved?

For most practical workloads: yes. A 128K-token request that once required 30+ minutes of prefill and 32 GB of KV cache now runs at practical latency on standard clusters. The 1M-token frontier is still being pushed.

References

- Liu et al., Ring Attention with Blockwise Transformers for Near-Infinite Context, ICLR 2024 · code
- Brandon et al., Striped Attention: Faster Ring Attention for Causal Transformers, 2023
- Yuan et al., Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, DeepSeek 2025
- Qiao et al., SwiftKV: Fast Prefill-Optimized Inference with Knowledge-Preserving Model Transformation, 2024 · Arctic Inference plugin
- Xiao et al., Efficient Streaming Language Models with Attention Sinks, ICLR 2024
- LMSYS, Chunked Pipeline Parallelism, Blog January 2026
- vLLM Context Parallel Deployment Docs
- vLLM engineering discussion: #22693 vs #34018 (Ring vs All-to-All CP)

Thank You

Questions?

Long-Context Inference · University of Tübingen · Summer 2026

Kavyanjali Agnihotri (Matriculation Number: 6975801)